

Университет ИТМО

Факультет программной инженерии и компьютерной техники

Лабораторная работа №1

по дисциплине «Системное программирование»

Выполнил:

Студент группы Р4114

Рыбин В. А.

Санкт-Петербург

2025

Задание

Использовать средство синтаксического анализа по выбору, реализовать модуль для разбора текста в соответствии с языком по варианту. Реализовать построение по исходному файлу с текстом синтаксического дерева с узлами, соответствующими элементам синтаксической модели языка. Вывести полученное дерево в файл в формате, поддерживающем просмотр графического представления.

Порядок выполнения:

1 Изучить выбранное средство синтаксического анализа

a. Средство должно поддерживать программный интерфейс, совместимый с языком Си

b. Средство должно параметризоваться спецификацией, описывающей синтаксическую структуру разбираемого языка

c.

Средство может функционировать посредством кодогенерации и/или подключения необходимых для его работы дополнительных библиотек

d. Средство может быть реализовано с нуля, в этом случае оно должно использовать обобщённый алгоритм, управляемый спецификацией

2 Изучить синтаксис разбираемого по варианту языка и записать спецификацию для средства

синтаксического анализа, включающую следующие конструкции:

a. Подпрограммы со списком аргументов и возвращаемым значением

b. Операции контроля потока управления – простые ветвления if-else и циклы или аналоги

c.

В зависимости от варианта – определения переменных

d. Целочисленные, строковые и односимвольные литералы

e. Выражения численной, битовой и логической арифметики

f.

Выражения над одномерными массивами

g.

Выражения вызова функции

3 Реализовать модуль, использующий средство синтаксического анализа для разбора языка по варианту

a. Программный интерфейс модуля должен принимать строку с текстом и возвращать структуру,

описывающую соответствующее дерево разбора и коллекцию сообщений ошибке

b. Результат работы модуля – дерево разбора – должно содержать иерархическое представление для всех синтаксических конструкций, включая выражения, логически представляющие собой иерархически организованные данные, даже если на уровне средства

синтаксического анализа для их разбора было использовано линейное представление

4 Реализовать тестовую программу для демонстрации работоспособности созданного модуля

а. Через аргументы командной строки программа должна принимать имя входного файла для

чтения и анализа, имя выходного файла записи для дерева, описывающего синтаксическую

структуру разобранного текста

б. Сообщения об ошибке должны выводиться тестовой программой (не модулем, отвечающим

за анализ!) в стандартный поток вывода ошибок

5 Результаты тестирования представить в виде отчета, в который включить:

а. В части 3 привести описание структур данных, представляющих результат разбора текста (3а)

б. В части 4 описать, какая дополнительная обработка потребовалась для результата разбора,

предоставляемого средством синтаксического анализа, чтобы сформировать результат работы

созданного модуля

с.

В части 5 привести примеры исходных анализируемых текстов для всех синтаксических

конструкций разбираемого языка и соответствующие результаты разбора

Грамматика

3 вариант

```
source: sourceItem*;
typeRef: {
|builtin: 'bool'|'byte'|'int'|'uint'|'long'|'ulong'|'char'|'string';
|custom: identifier;
|array: typeRef '(' (','*) ')';
};
funcSignature: identifier '(' list<argDef> ')' ('as' typeRef)? {
argDef: identifier ('as' typeRef)?;
};
sourceItem: {
|funcDef: 'function' funcSignature statement* 'end' 'function';
};
statement: {
|var: 'dim' list<identifier> 'as' typeRef; // for static typing
|if: 'if' expr 'then' statement* ('else' statement*)? 'end' 'if';
|while: 'while' expr statement* 'wend';
|do: 'do' statement* 'loop' ('while'|'until') expr;
```

```

|break: 'break';
|expression: expr ';';
};
expr: { // присваивание через '='
|binary: expr binOp expr; // где binOp - символ бинарного оператора
|unary: unOp expr; // где unOp - символ унарного оператора
|braces: '(' expr ')';
|callOrIndexer: expr '(' list<expr> ')';
|place: identifier;
|literal: bool|str|char|hex|bits|dec;
};

```

Bison (грамматика)

```

%{

#define YYERROR_VERBOSE

#include <stdio.h>
#include "node.h"

extern int yylineno;

int yylex();
void yyerror(struct Node **node, const char *s);

%}

%locations
%define parse.error verbose

%union {
    struct Node *node;
}

%parse-param { struct Node **root }

%token <node> COMMA;
%token <node> nativee
%token <node> identifier
%token <node> STR
%token <node> DEC
%token <node> CHAR
%token <node> HEX
%token <node> BITS

%left PLUS
%left MINUS
%left MULTIPLY
%left DIVIDE
%left MORE
%left LESS
%left OR
%left AND
%right EQ
%right NOT_EQ

%token FUNCTION
%token END
%token DIM
%token AS
%token BR_CLOSE
%token BR_OPEN
%token SEMICOLON
%token ASSIGNMENT
%token <node> BREAK
%token <node> BOOL
%token WEND
%token <node> WHILE
%token DO
%token LOOP

```

```

%token <node> UNTIL
%token IF
%token THEN
%token ELSE
%token NOT

%type <node> typeRef
%type <node> typeRefOpt
%type <node> argDef
%type <node> custom
%type <node> builtin
%type <node> start
%type <node> funcSignature
%type <node> list
%type <node> listOpt
%type <node> listIdentifier
%type <node> listIdentifierOpt
%type <node> statement
%type <node> var
%type <node> listStatment
%type <node> funcDef
%type <node> expression
%type <node> literal
%type <node> expr
%type <node> assignment
%type <node> place
%type <node> array
%type <node> binary
%type <node> braces
%type <node> break
%type <node> commas
%type <node> while
%type <node> whileOrUntil
%type <node> do
%type <node> if
%type <node> elseBlockOpt
%type <node> sourceItem
%type <node> listSourceItem
%type <node> elseBlock
%type <node> unary
%type <node> listExpr
%type <node> callOrIndexer

%%

start: listSourceItem { *root = $1; $$ = $1; }
;

typeRefOpt: { $$ = create_node(0, "", "VoidType"); }
| AS typeRef { $$ = $2; }
;

typeRef: custom { $$ = $1; }
| builtin { $$ = $1; }
| array { $$ = $1; }
;

array: typeRef BR_OPEN commas BR_CLOSE
{
    $$ = create_node(2, "", "Array");
    add_child($$, $1, 0);
    add_child($$, $3, 1);
}

commas: commas COMMA { $$ = create_node(2, "", "PartOfArray"); add_child($$, $1, 0); add_child($$, create_node(0, "", "EmptyNode"), 1); }
| { $$ = NULL; }
;

argDef: identifier typeRefOpt
{
    if ($2 != NULL)
    {
        $$ = create_node(2, "", "argDef");

        add_child($$, $1, 0);
        add_child($$, $2, 1);
    }
    else
    {
        $$ = create_node(1, "", "argDef");
        add_child($$, $1, 0);
    }
}
;

```

```

listOpt: { $$ = create_node(0, "", "EmptyListOpt"); }
        | list { $$ = $1; }
;

list: list COMMA argDef { $$ = create_node(2, "", "ArgDefList"); add_child($$, $1, 0); add_child($$, $3, 1); }
    | argDef { $$ = $1; }
;

listSourceItem: listSourceItem sourceItem { $$ = create_node(2, "", "SourceItemList"); add_child($$, $1, 1); add_child($$, $2, 0); }
              | { $$ = NULL; }
;

sourceItem: funcDef { $$ = $1; }
;

funcSignature: identifier BR_OPEN listOpt BR_CLOSE typeRefOpt { $$ = create_node(3, "", "FuncSignature"); add_child($$, $1, 0); add_child($$, $3, 1);
add_child($$, $5, 2); }
;

custom: identifier { $$ = $1; }
;

builtin: nativee { $$ = $1; }
;

listIdentifierOpt: /* empty */ { { $$ = NULL; } }
                | listIdentifier { { $$ = $1; } }
;

listIdentifier: listIdentifier COMMA identifier { $$ = create_node(2, "", "Variables"); add_child($$, $1, 0); add_child($$, $3, 1); }
              | identifier { $$ = $1; }
;

funcDef: FUNCTION funcSignature listStatment END FUNCTION { $$ = create_node(2, "", "FuncDef"); add_child($$, $2, 0); add_child($$, $3, 1); }
;

listStatment: statement listStatment { $$ = create_node(2, "", "ListStatement"); add_child($$, $2, 1); add_child($$, $1, 0); }
            | { $$ = NULL; }
;

statement: var { $$ = $1; }
          | if { $$ = $1; }
          | while { $$ = $1; }
          | do { $$ = $1; }
          | break { $$ = $1; }
          | expression { $$ = $1; }
;

if: IF expr THEN listStatment elseBlockOpt END IF
  {
    $$ = create_node(3, "", "If");

    add_child($$, $2, 0);
    add_child($$, $4, 1);
    add_child($$, $5, 2);
  }
  | IF expr error listStatment elseBlockOpt END IF { $$ = NULL; }
;

elseBlockOpt: { $$ = NULL; }
            | elseBlock { $$ = $1; }
;

elseBlock: ELSE listStatment { $$ = create_node(1, "", "ElseBlock"); add_child($$, $2, 0); }
;

while: WHILE expr listStatment WEND
      {
        $$ = create_node(3, "", "While");

        add_child($$, $1, 0);
        add_child($$, $2, 1);
        add_child($$, $3, 2);
      }
;

whileOrUntil: WHILE { $$ = $1; }
            | UNTIL { $$ = $1; }
;

do: DO listStatment LOOP whileOrUntil expr
  {
    $$ = create_node(3, "", "Do");
    add_child($$, $2, 0);
    add_child($$, $4, 1);
    add_child($$, $5, 2);
  }
;

```

```

break: BREAK { $$ = $1; }

var: DIM listIdentifier AS typeRef
{
    $$ = create_node(2, "", "Var");
    add_child($$, $2, 0);
    add_child($$, $4, 1);
}

;

expression: expr SEMICOLON { $$ = $1; }
| expr error { $$ = $1; }

;

expr: assignment { $$ = $1; }
| binary { $$ = $1; }
| unary { $$ = $1; }
| braces { $$ = $1; }
| callOrIndexer { $$ = $1; }
| place { $$ = $1; }
| literal { $$ = $1; }

;

assignment: expr ASSIGNMENT expr
{
    $$ = create_node(2, "", "Assignment");
    add_child($$, $1, 0);
    add_child($$, $3, 1);
}

;

binary: expr PLUS expr { $$ = create_node(2, "", "Plus"); add_child($$, $1, 0); add_child($$, $3, 1); }
| expr MINUS expr { $$ = create_node(2, "", "Minus"); add_child($$, $1, 0); add_child($$, $3, 1); }
| expr MULTIPLY expr { $$ = create_node(2, "", "Multiply"); add_child($$, $1, 0); add_child($$, $3, 1); }
| expr DIVIDE expr { $$ = create_node(2, "", "Divide"); add_child($$, $1, 0); add_child($$, $3, 1); }
| expr EQ expr { $$ = create_node(2, "", "Equals"); add_child($$, $1, 0); add_child($$, $3, 1); }
| expr NOT_EQ expr { $$ = create_node(2, "", "NotEquals"); add_child($$, $1, 0); add_child($$, $3, 1); }
| expr MORE expr { $$ = create_node(2, "", "More"); add_child($$, $1, 0); add_child($$, $3, 1); }
| expr LESS expr { $$ = create_node(2, "", "Less"); add_child($$, $1, 0); add_child($$, $3, 1); }
| expr OR expr { $$ = create_node(2, "", "Or"); add_child($$, $1, 0); add_child($$, $3, 1); }
| expr AND expr { $$ = create_node(2, "", "And"); add_child($$, $1, 0); add_child($$, $3, 1); }

;

unary: PLUS expr { $$ = create_node(1, "", "UnaryPlus"); add_child($$, $2, 0); }
| MINUS expr { $$ = create_node(1, "", "UnaryMinus"); add_child($$, $2, 0); }
| NOT expr { $$ = create_node(1, "", "Not"); add_child($$, $2, 0); }

;

braces: BR_OPEN expr BR_CLOSE { $$ = $2; }

;

listExpr:
{
    $$ = NULL; }
| listExpr COMMA expr
{
    $$ = create_node(2, "", "ListExpr");
    add_child($$, $1, 0);
    add_child($$, $3, 1);
}
| expr { $$ = $1; }

;

callOrIndexer: expr BR_OPEN listExpr BR_CLOSE
{
    $$ = create_node(2, "", "CallOrIndexer");
    add_child($$, $1, 0);
    add_child($$, $3, 1);
}

;

place: identifier { $$ = $1; }

;

literal: BOOL { $$ = $1; }
| STR { $$ = $1; }
| CHAR { $$ = $1; }
| HEX { $$ = $1; }
| DEC { $$ = $1; }
| BITS { $$ = $1; }

;

%%

void yyerror(struct Node **root, const char *s) {
    printf("Error: %s. Line %d\n", s, yylineno - 1);
}

```

Flex (лексер)

```
%{
#include "parser.tab.h"
#include "node.h"

%}

%option yylineno
%option noyywrap
%%

int { struct Node *node = create_node(0, "", "int"); yylval.node = node; return nativee; }

uint { struct Node *node = create_node(0, "", "uint"); yylval.node = node; return nativee; }

long { struct Node *node = create_node(0, "", "long"); yylval.node = node; return nativee; }

ulong { struct Node *node = create_node(0, "", "ulong"); yylval.node = node; return nativee; }

char { struct Node *node = create_node(0, "", "char"); yylval.node = node; return nativee; }

string { struct Node *node = create_node(0, "", "string"); yylval.node = node; return nativee; }

byte { struct Node *node = create_node(0, "", "byte"); yylval.node = node; return nativee; }

bool { struct Node *node = create_node(0, "", "bool"); yylval.node = node; return nativee; }

as { return AS; }

dim { return DIM; }

end { return END; }

break { struct Node *node = create_node(0, "", "Break"); yylval.node = node; return BREAK; }

function { return FUNCTION; }

\[^[^"\\]*(?:\\\[^\\" data-bbox="115 900 160 911">{
```



```

    struct Node *node = create_node(0, yytext, "Char");
    yylval.node = node;
    return CHAR;
}

[a-zA-Z_][a-zA-Z_0-9]* {
    struct Node *node = create_node(0, yytext, "Identifier");
    yylval.node = node;
    return identifier;
}

[0-9]+ {
    struct Node *node = create_node(0, yytext, "Number");
    yylval.node = node;
    return DEC;
}

\" {return BR_OPEN; }
\\ {return BR_CLOSE;}
\\, { return COMMA; }

\\+ { return PLUS; }
\\- { return MINUS; }
\\* { return MULTIPLY; }
\\/ { return DIVIDE; }
\\! { return NOT; }

[ \\t\\n] {}

%%

```

Описание структур данных

Для построения дерева была реализована структура данных Node.

```

struct Node {

    char *text;
    char *type;

    size_t id;
    size_t children_amount;
    struct Node **children;
};

```

Вывод:

Было изучено, как работает парсер + лексер. На практике был использован Bison + Flex. Также было повторено, как работать с Makefile.